SLIDE 01

Agent Orchestration

Hi, I'm Sergei Telegin. Today, I'll show you one real task from my AI development process. We'll look at what the agents checked, why they had to repeat some steps, and how much time and how many tokens this took.

SLIDE 02

157 tests passed. QA blocked the PR.

TrafficRulesApp is my personal iOS app for the Spanish driving theory exam. In this task, the agents added tests. The reviewer approved the changes. All 157 tests passed. But QA still blocked the pull request.

Some tests passed without really proving that the code worked correctly.

We'll follow the task on one diagram. We'll look at the problem, the fix, and the cost. As we go, think about this question: what should each stage pass to the next one?

SLIDE 03

A little theory: an agent is a loop

First, how does a single agent usually work?

The model gets some context and chooses an action. For example, it can use a tool to read a file, edit code, or run a test. The system around the model runs that tool and returns the result. This system is often called a harness.

The model then chooses what to do next. It might check the code, make a change, see a test failure, and try again. This is the loop.

Now, how do several agents work together? Someone must give them tasks and decide when the next stage can start. My orchestrator does this by following written instructions. The diagram shows these rules, but it does not make the agents follow them automatically.

SLIDE 04

One map, four responsibilities

Here is the map we'll use. It has four main roles: planner, developer, reviewer, and QA.

The planner breaks the request into tasks and defines acceptance criteria: the conditions we use to decide whether the work is complete.

The developer writes the code and reports what it changed and tested. The reviewer compares the code changes with the requirements. QA runs the tests and checks that the result meets those requirements.

These agents have separate contexts, but they can still make the same mistakes. They may use the same model and make the same assumptions. Each role is useful because it adds another check.

The orchestrator manages the whole process. It passes the task, project profile, and earlier results between stages. It also clears up questions, keeps the Git branch separate from other work, checks the results, and updates the issues.

Look at the labels between the boxes. Each stage gives the next one something it can check. Just saying "done" is not enough.

SLIDE 05

Changed code needs fresh evidence

The work can go back to the developer after review or after QA. But when the developer fixes a QA problem, the reviewer must check the fix before QA tests it again.

The reason is simple: the code has changed. Approval of the old version does not automatically cover the new one.

There is a short story behind this rule. In July, QA found that session time could be counted twice, even though the reviewer had approved PR 143. On August 26, a commit added rule eleven: review QA fixes before testing them again.

We know when the rule changed. We cannot say that the July bug was the only reason, or that we know every setting from those older sessions.

Today, each fix loop has a limit of three rounds. If problems remain, the orchestrator stops and asks for help. These limits are also written instructions. Let's look at a later run where we can see the extra review in the reports.

SLIDE 06

32 new tests. One delivery route.

The goal of PR 198 was to add unit tests for ProgressStore and for saving progress when a test session ends. This included checking that session time was not counted twice. So the task was connected to the earlier bug.

The change added 32 tests. The total grew from 125 to 157. It also added a project profile with commands and rules for this project. The application code stayed the same.

These are saved reports and screenshots from that run. I did not run the tests again for this talk. The map shows the planned process. The reports show the steps that actually happened.

The goal was to test the existing behavior properly. Adding more tests was not enough. A useful test should fail when the behavior it checks is broken. Both review and QA had a part to play here.

SLIDE 07

Review improved the patch

The first reviewer gave conditional approval. It found no critical problems, but suggested two improvements.

One was a missing integration test for an important progress rule. The other was a check that the expected number of themes matched the data included in the app.

The developer made these changes. The second reviewer checked them, ran the tests, and approved the result.

Review had improved the tests. QA then found a different problem.

On the map, we have reached the review stage twice. Each review checked a different version of the code. After the second approval, the task moved to QA.

SLIDE 08

Green tests. Weak evidence.

QA ran the tests on two simulators. It also ran the full test suite without parallel testing. The tests passed, but QA still reported BLOCKED because it found three problems within the task's scope.

Two of these problems were connected. Five tests checked that the total session time was greater than zero. They also used shared storage. Their setup code deleted a fixed list of stored values.

Each action that ended a session should have added time. But a total greater than zero did not prove that this action had added anything. An old value could make the test pass.

The third problem was about checking two different exit paths. We can discuss that during questions.

This does not mean that all 157 tests were bad. QA pointed out the weak checks and also noted the ones that worked well. A test can pass and still fail to check what we need. Let me show you an example.

SLIDE 09

Prove the change, isolate the state

Imagine there are 500 seconds in storage before we call the method. The method adds nothing. After the call, there are still 500 seconds.

The check "Is the total greater than zero?" passes. The check "Did the total increase?" fails.

These checks answer different questions. The number 500 is just an example. It is not a result from this test run.

Why could an old value be there? The tests used UserDefaults.standard, which is shared application storage. They deleted a known list of keys, but other stored values could still affect the starting state. The reports describe this problem during development.

The fix had two parts. First, each test got its own temporary storage. The application already supported passing in a different store, so no application changes were needed.

Second, each test saved the value before the action and checked that it increased afterward.

Separate storage gives the test a known starting state. The before-and-after check shows whether the action added time. We need to consider both the setup and the check.

There is one important detail. In the fixed tests, the starting value is zero. So "greater than zero" and "greater than the previous value" give the same result. We cannot say that changing the comparison alone solved the problem.

The developer only changed the test files. But those changes still needed another review before QA could test them again.

SLIDE 10

Fix, review, QA

The developer pushed the fixes to the same pull request. The third review followed rule eleven and checked the fixes before they went back to QA.

The reviewer approved them, but also pointed out the detail we just discussed: with separate storage, the starting value is zero. The before-and-after check describes what we want to test. Separate storage removes the problem with old shared values.

The reviewer checked both the code and our explanation of why the fix worked.

QA then reported GREEN. All 157 tests passed in each of four full runs. One run deliberately put old values into shared storage to check that the tests were separate from it.

When the screenshots were taken on September 6, the PR was still open. Review and QA had passed, but the PR had not been merged. In this process, green QA is followed by final checks and cleanup.

Now, let's look at the cost of these steps.

SLIDE 11

The observed cost

The report shows nine subagent launches and about 1.1 million subagent tokens.

Exact figure on the slide: 1,097,667 subagent tokens.

The time recorded for all stages adds up to about one hour and thirty-one minutes. This is the sum of the stage times, not a measurement of all the time I spent on the task.

There was one planning stage, three developer rounds, three reviews, and two QA rounds. Review used slightly more tokens than development.

The orchestrator's tokens were part of the main conversation. They were estimated separately and are not included here. I'm also leaving out estimated costs in dollars.

Next, I would measure how many real problems the agents found, how many they missed, and how much time a person spent helping them. Together with tokens and time, this would help us decide which checks are useful.

SLIDE 12

How the repository works today

Since that run, I have changed how the files are organized. The shared instructions now live in skills. The orchestrator stays in the main conversation. It can ask for missing details and give work to separate agents.

Small adapters define how to start agents and choose settings in each system. We can choose a model and reasoning settings for each role if the system supports this. Otherwise, it uses its default settings.

A skill is a set of instructions. Loading a skill does not start a separate agent by itself.

The project profile contains commands and project rules. Each role also reads the relevant shared project memory and follows common rules for using it.

This makes the purpose of each file clearer. But the run I showed happened before these changes. I have not measured whether this new structure improves cost or quality.

SLIDE 13

Define what each stage hands over

You can use this idea without copying my whole process: define what each stage must give to the next one.

The planner should provide clear tasks, conditions we can check, and dependencies between tasks.

The developer needs those requirements, project information, and rules about what it can do. It returns a pull request, the code diff, the version it changed, and the build and test results.

The reviewer needs the actual code changes and requirements, not just the developer's summary. Its report should name the version it checked and list problems by importance, or give clear approval. After a fix, it also needs the earlier findings to check whether they were addressed.

QA needs the same requirements and the current code. It shows how each requirement was checked and gives the results. It also reports any bugs, explains their type, and gives a clear GREEN or BLOCKED result. If we say a bug already existed, we must show it in the version before our changes. If we are unsure, we cannot simply mark the work green.

The orchestrator passes these results on and keeps the rules for the task. These include the branch, who is responsible for each part, the working mode, and limits on changes. In dry-run mode, the agents prepare local plans and reports instead of publishing issues or pull requests.

Today, written instructions manage most of this. In the future, software could check that the required files and reports exist and match the current version. It could reject approval for an older version. But that still would not prove that the review was correct.

SLIDE 14

Three things to take back

Here are three things to take back.

First, define the result of each stage. Say what it must produce and when the next stage can begin.

Second, pass results that others can check: requirements, code changes, review findings, and test results. Give the next role something concrete to work with.

Third, repeat the checks after code changes. A fix needs another review before QA tests it again.

Start with one type of task and look at how the agents pass work between stages. The repository contains the skills, profiles, process, and examples we discussed.